
```
def myFunction(param1, param2, param3):  
    # body of function  
  
# call to a function:  
myFunction(argument1, argument2, argument3)
```

The same rule applies with methods and method calls. However, you may notice that whenever we make a call to a method, it appears that we are specifying one less argument than the number of parameters. For example, the definition of the `power()` method in our `TV` class looks like this:

```
def power(self):
```

This implies that the `power()` method is expecting one value to be passed in and whatever is passed in will be assigned to the variable `self`. Yet when we started by turning on the TV in [Listing 2-8](#), we made this call:

```
oTV.power()
```

When we make the call, we don't explicitly pass anything inside the parentheses.

This may seem even stranger in the case of the `setChannel()` method. The method is written to accept two parameters:

```
def setchannel(self, newchannel):  
    if newChannel in self.channelList:  
        self.channelIndex = self.channelList.index(newChannel)
```

But we called `setChannel()` like this:

```
oTV.setChannel(11)
```

It appears that only one value is being passed in.

You might expect Python to generate an error here, due to a mismatch in the number of arguments (one) and the number of parameters (two). In practice, Python is doing a bit of behind-the-scenes work to make the syntax easier to follow.